Engineering Principles

Version: 1.0
Status: Approved
Owner: Engineering Team
Applies To: All CDIS Engineering Projects

Purpose

The Engineering Principles define the core values that guide every architectural and technical decision made within CDIS. These principles ensure that all engineers build software consistently, securely, and maintainably while providing an excellent developer experience.

These principles are **not coding standards** and **not implementation guidelines**. Instead, they represent the philosophy behind every engineering decision.

Whenever multiple technical solutions are available, engineers should evaluate them against these principles before making a decision.

Engineering Philosophy

Our goal is not simply to deliver software.

Our goal is to build software that is:

- Easy to understand
- Easy to maintain
- Easy to extend
- Secure by default
- Consistent across all projects
- Productive for every engineer

Every project developed within CDIS should feel familiar, regardless of the engineer or team that built it.

Core Engineering Principles

1. Developer Experience First

Principle

A new engineer should be able to clone, configure, understand, and start contributing to a project with minimal guidance.

Why

Developer productivity directly impacts project delivery, onboarding time, maintenance cost, and engineering efficiency.

Guidelines

- One-command project setup whenever possible.
- Clear project structure.
- Well-documented development workflow.
- Minimal manual configuration.
- Consistent tooling across all projects.

---

2. Convention Over Configuration

Principle

The engineering platform should provide one recommended way of building software instead of allowing multiple equally valid approaches.

Why

Reducing unnecessary choices improves consistency, simplifies onboarding, and reduces maintenance.

Examples

Instead of allowing multiple naming conventions:

```
userCard.tsx
UserCard.jsx
user-card.tsx
```

Use one standard:

```
UserCard.tsx
```

Instead of multiple folder structures, every project should follow the same organization.

---

3. Consistency Over Personal Preference

Principle

Engineering decisions should prioritize consistency across the organization rather than individual developer preferences.

Why

Every project should look and feel familiar regardless of who developed it.

Consistent projects are easier to:

- Review
- Maintain
- Debug
- Extend
- Transfer between teams

---

4. Secure by Default

Principle

Security should be built into the platform rather than relying on developers to implement it correctly every time.

Why

Security mistakes are easier to prevent than to fix after deployment.

Examples

Projects should include secure defaults such as:

- Authentication support
- Authorization framework
- Input validation
- Secure HTTP headers
- Environment variable management
- Password hashing
- Protection against common web vulnerabilities

---

5. Simplicity Over Cleverness

Principle

Prefer solutions that are easy to understand over solutions that are unnecessarily clever or complex.

Why

Software is read significantly more often than it is written.

Simple systems are easier to:

- Maintain
- Debug
- Extend
- Review
- Onboard new engineers

Whenever multiple solutions satisfy the same requirements, choose the simplest maintainable solution.

Supporting Engineering Practices

The following practices support the core principles.

Reuse Before Rebuild

Before creating new functionality, determine whether an existing implementation can be reused.

Examples include:

- Components
- Hooks
- Services
- Utilities
- API clients
- Layouts

Reducing duplication improves maintainability and consistency.

Modular by Design

Applications should be organized into independent modules with clear responsibilities.

Modules should have:

- High cohesion
- Low coupling
- Clear interfaces
- Minimal dependencies

This enables easier testing, maintenance, and future expansion.

Documentation is Part of the Product

Documentation is considered part of the implementation.

Every significant engineering decision should be documented.

Documentation should explain:

- What was implemented
- Why it was implemented
- How it should be used

Code without documentation increases long-term maintenance costs.

Production Ready by Default

Every new project should include the essential engineering infrastructure required for production deployment.

Examples include:

- Logging
- Error handling
- Environment configuration
- Code formatting
- Linting
- Testing framework
- Docker support
- API documentation
- CI/CD readiness

Projects should not require engineers to repeatedly configure common production features.

Build Once, Use Everywhere

The engineering platform should serve as the foundation for all future applications.

Whenever improvements are made to the platform, future projects should benefit from those improvements rather than reimplementing similar functionality.

Decision-Making Framework

When evaluating multiple technical approaches, engineers should ask the following questions:

1. Does this improve the developer experience?
2. Does this maintain organizational consistency?
3. Is it secure by default?
4. Is it simpler than the alternatives?
5. Can it be reused in future projects?
6. Will another engineer easily understand this decision?

If the answer to most of these questions is **No**, reconsider the proposed solution.

Expected Outcomes

Following these principles should result in:

- Faster onboarding
- Consistent project architecture
- Reduced maintenance effort
- Improved software quality
- Secure engineering practices
- Easier collaboration between teams
- Faster project delivery
- Higher long-term engineering productivity

Scope

These principles apply to:

- Frontend applications
- Backend services
- APIs
- Shared libraries

- Internal tools
- Engineering templates
- Starter toolkits
- Future engineering platforms

References

- The Pragmatic Programmer
- Clean Architecture — Robert C. Martin
- A Philosophy of Software Design — John Ousterhout
- Google Engineering Practices
- The Twelve-Factor App Methodology
- OWASP Secure Coding Practices
- Conventional Commits Specification
- Semantic Versioning (SemVer)